

从 schedule() 到 switch_to: Linux 3.18.6 的调度时机与任务切换

课程：《Linux 内核原理与分析》

实验：实验八（第九周）

姓名：叶丁再（真实姓名须与最后申请证书的姓名一致）

学号：20262809

日期：2026 年 9 月 27 日

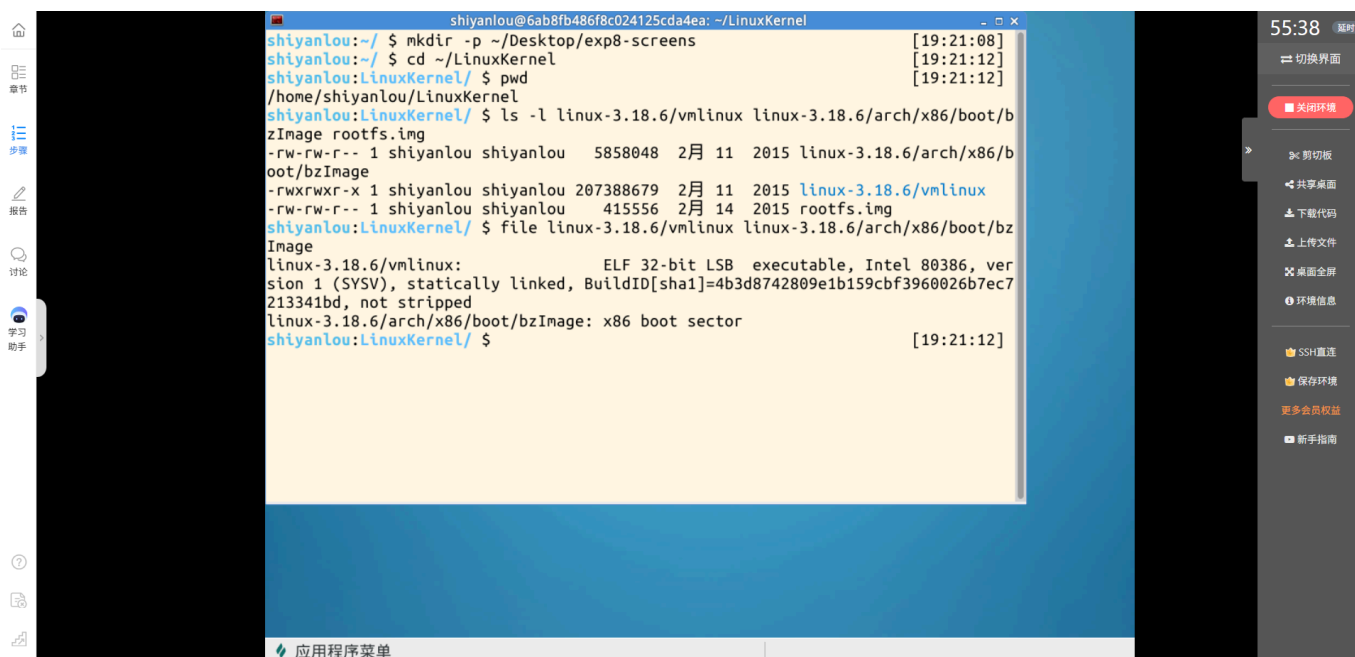
原创作品转载请注明出处：叶丁再

课程：《Linux 内核分析》MOOC 课程 <http://mooc.study.163.com/course/USTC-1000029000>

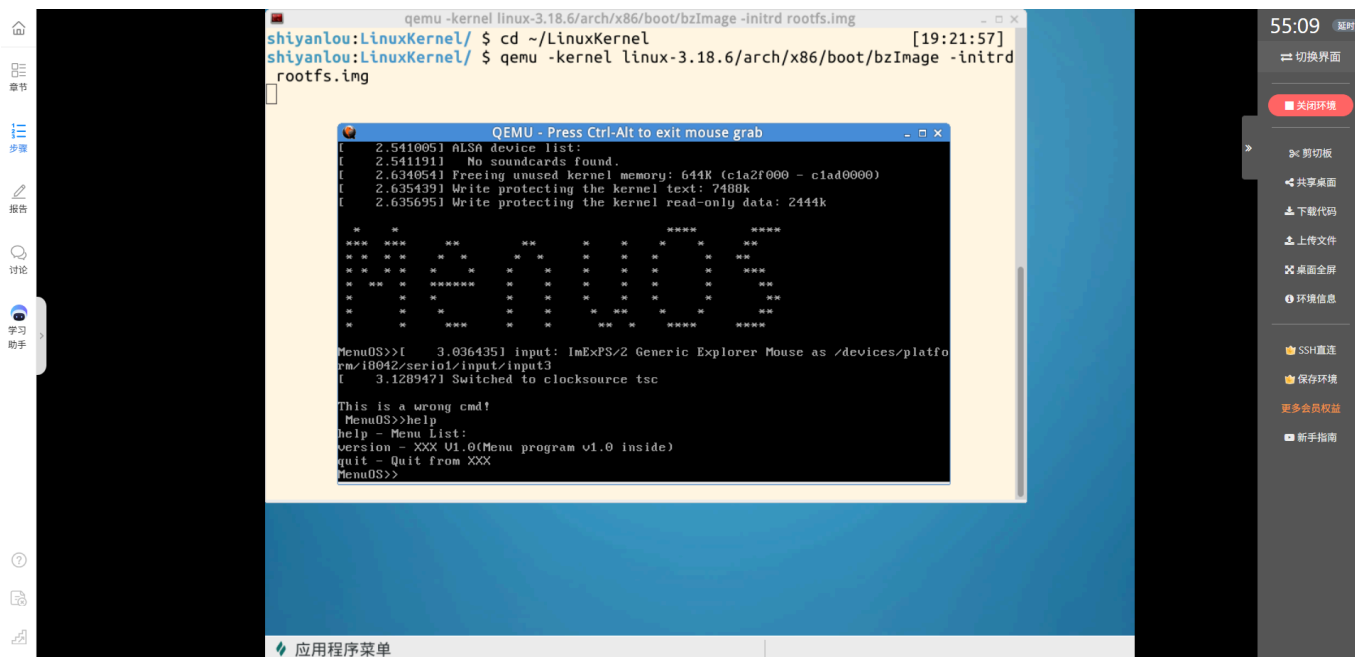
一、实验目标与环境

本次实验关注三个问题：Linux 在什么情况下进入调度器；调度器选择另一个任务后怎样切换 CPU 的内核执行现场；这种任务切换与系统调用/中断入口保存的现场是什么关系。

云端环境使用 Linux 3.18.6、x86 32 位内核、QEMU 和 GDB。环境检查截图显示 `vmlinux` 是 32 位 Intel 80386 ELF，`bzImage` 与 `rootfs.img` 均存在。每个课程实验会重置云端环境，因此本周启动的 MenuOS 没有实验六添加的 `fork` 命令；`help` 只列出版本和退出命令。本次没有把旧实验的 MenuOS 改动当作当前证据，也没有为调度跟踪重编译它。



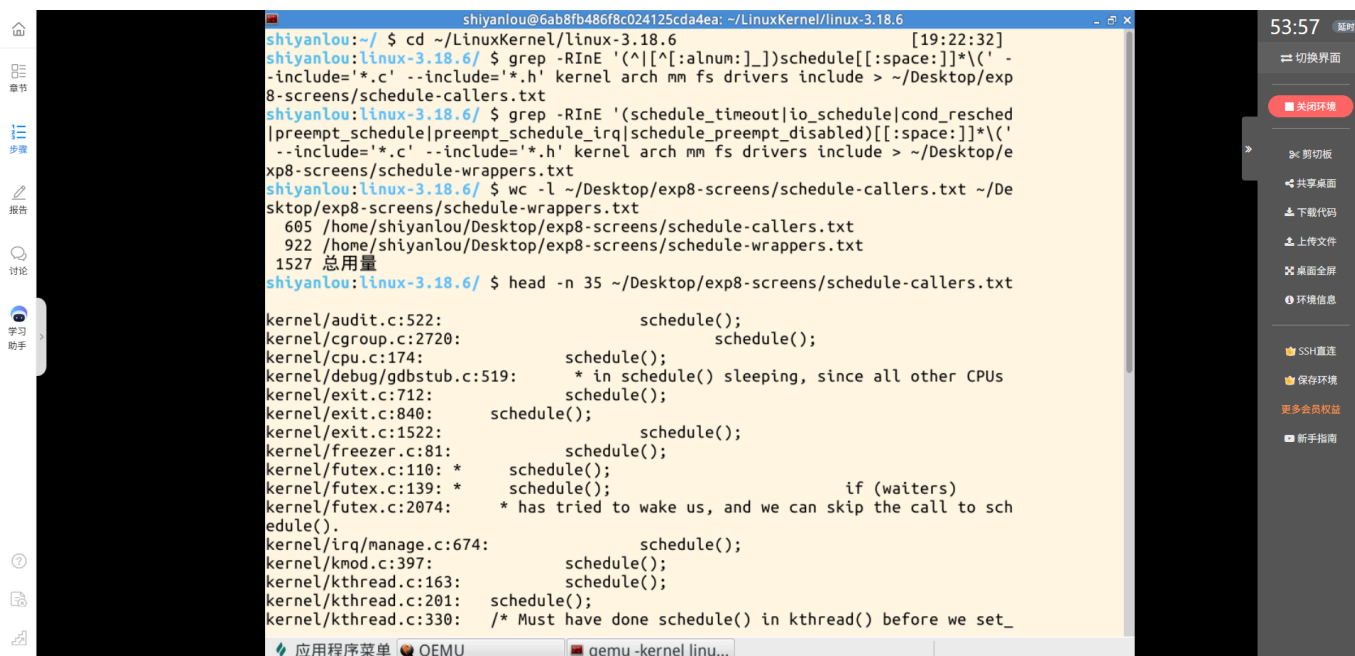
```
shiyanolou@6ab8fb486f8c024125cda4ea: ~/LinuxKernel
shiyanolou:~/ $ mkdir -p ~/Desktop/exp8-screens [19:21:08]
shiyanolou:~/ $ cd ~/LinuxKernel [19:21:12]
shiyanolou:LinuxKernel/ $ pwd [19:21:12]
/home/shiyanolou/LinuxKernel
shiyanolou:LinuxKernel/ $ ls -l linux-3.18.6/vmlinux linux-3.18.6/arch/x86/boot/bzImage rootfs.img
-rw-rw-r-- 1 shiyanolou shiyanolou 5858048 2月 11 2015 linux-3.18.6/arch/x86/boot/bzImage
-rwxrwxr-x 1 shiyanolou shiyanolou 207388679 2月 11 2015 linux-3.18.6/vmlinux
-rw-rw-r-- 1 shiyanolou shiyanolou 415556 2月 14 2015 rootfs.img
shiyanolou:LinuxKernel/ $ file linux-3.18.6/vmlinux linux-3.18.6/arch/x86/boot/bzImage
linux-3.18.6/vmlinux: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), statically linked, BuildID[sha1]=4b3d8742809e1b159cbf3960026b7ec7213341bd, not stripped
linux-3.18.6/arch/x86/boot/bzImage: x86 boot sector
shiyanolou:LinuxKernel/ $ [19:21:12]
```



由于当前 MenuOS 没有自定义进程创建命令，我从内核启动时的调度路径开始跟踪。这样可以直接观察现有内核中的任务切换，不依赖实验之间不会保留的用户态命令。

二、搜索 schedule()：调用命中只是候选清单

我在 Linux 3.18.6 源码树中搜索 `schedule()` 以及 `schedule_timeout()`、`io_schedule()`、`cond_resched()`、`preempt_schedule()` 等调度封装。终端显示第一组表达式得到 605 行，第二组得到 922 行。



```

shiyaniou@6ab8fb486f8c024125cda4ea: ~/LinuxKernel/linux-3.18.6
kernel/futex.c:139: *      schedule();                                if (waiters)
kernel/futex.c:2074: * has tried to wake us, and we can skip the call to sch
edule().
kernel/irq/manager.c:674:      schedule();
kernel/kmod.c:397:      schedule();
kernel/kthread.c:163:      schedule();
kernel/kthread.c:201:      schedule();
kernel/kthread.c:330: /* Must have done schedule() in kthread() before we set_
task_cpu */
kernel/kthread.c:498:      schedule();
kernel/kthread.c:578:      schedule();
kernel/locking/rtmutex-tester.c:176:      schedule();
kernel/locking/rtmutex-tester.c:225:      schedule();
kernel/locking/rtmutex-tester.c:254:      schedule();
kernel/locking/rtmutex.c:1152:      schedule();
kernel/locking/rtmutex_common.h:19: * call schedule_rt_mutex_test() instead of s
chedule() for the tasks which
kernel/locking/rtmutex_common.h:30:      schedule();                                \
kernel/locking/rtmutex_common.h:36:# define schedule_rt_mutex(_lock)          s
chedule()
kernel/locking/rwsem-spinlock.c:153:      schedule();
kernel/locking/rwsem-spinlock.c:213:      schedule();
kernel/locking/rwsem-xadd.c:242:      schedule();
kernel/locking/rwsem-xadd.c:458:      schedule();
kernel/pid_namespace.c:240:      schedule();
kernel/power/process.c:245:      schedule();
kernel/power/process.c:266:      schedule();
kernel/printk/printk.c:141:/* Flag: console code may call schedule() */
kernel/profile.c:6: * schedule() calls via kernel command line parameters passe
d at boot.
kernel/rcu/tree.h:409:      schedule();                                \
shiyaniou:linux-3.18.6/ $
[19:23:23]

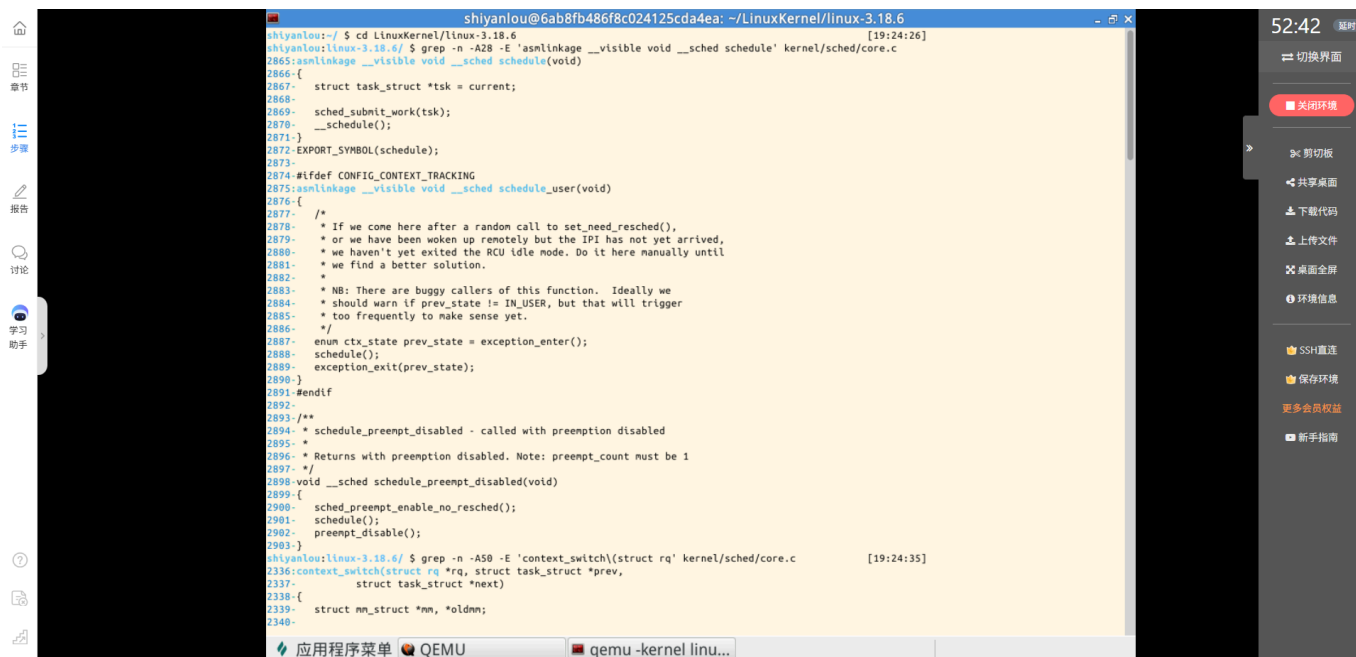
```

这些数字是 grep 对源码文本的匹配行数，不等于 605 次或 922 次实际调度。搜索还会匹配注释、定义、声明、宏以及条件编译代码；必须结合上下文区分“某处写着 schedule”与“本次运行真正调用了调度器”。这次保存的结果适合作为待检查清单，不能直接当成完整的已分类调用统计。

从源码和本次观察可以把调度相关时机概括为几类：

1. **任务主动等待或睡眠。**任务在等待资源、定时器或其他任务时改变自身状态，并沿等待路径调用 `schedule()` 或相关封装。调度器随后选择可运行任务。
2. **内核中的主动让出点。**`cond_resched()` 等封装允许内核在满足条件时检查是否需要重新调度；封装被调用不代表每次都会切换。
3. **重调度请求。**定时器 tick、任务唤醒或优先级变化可能设置 `need_resched`/任务标志。设置标志只是请求，内核要等到允许调度的安全位置再处理。
4. **系统调用或异常返回前。**x86 32 位的 `entry_32.S` 中，`work_pending` 检查 `TIF_NEED_RESCHED`；标志未设置时走其他返回工作，设置时才调用 `schedule`，之后还会重新检查标志。

本次源码截图里，`schedule()` 本身先调用 `sched_submit_work(tsk)`，再进入 `__schedule()`。这也说明调度入口和实际任务切换不是同一概念：调度器仍可能选择当前任务；只有 `prev` 和 `next` 不同，才会走到实际的上下文切换。



```

shiyaniou@6ab8fb486f8c024125cda4ea: ~/LinuxKernel/linux-3.18.6
shiyaniou:~/ $ cd LinuxKernel/linux-3.18.6
shiyaniou:linux-3.18.6/ $ grep -n -A28 -E 'asm\<div>__visible void __sched schedule(void)' kernel/sched/core.c
2865:asm\<div>__visible void __sched schedule(void)
2866-{
2867-    struct task_struct *tsk = current;
2868-
2869-    sched_submitt_work(tsk);
2870-    __schedule();
2871-}
2872-EXPORT_SYMBOL(schedule);
2873-
2874-#ifdef CONFIG_CONTEXT_TRACKING
2875:asm\<div>__visible void __sched schedule_user(void)
2876-{
2877-    /*
2878-     * If we come here after a random call to set_need_resched(),
2879-     * or we have been woken up remotely but the IPI has not yet arrived,
2880-     * we haven't yet exited the RCU idle mode. Do it here manually until
2881-     * we find a better solution.
2882-     *
2883-     * NB: There are buggy callers of this function. Ideally we
2884-     * should warn if prev_state != IN_USER, but that will trigger
2885-     * too frequently to make sense yet.
2886-     */
2887-    enum ctx_state prev_state = exception_enter();
2888-    schedule();
2889-    exception_exit(prev_state);
2890-}
2891-#endif
2892-
2893-/**
2894- * schedule_preempt_disabled - called with preemption disabled
2895- *
2896- * Returns with preemption disabled. Note: preempt_count must be 1
2897- */
2898-void __sched schedule_preempt_disabled(void)
2899-{
2900-    sched_preempt_enable_no_resched();
2901-    schedule();
2902-    preempt_disable();
2903-}
shiyaniou:linux-3.18.6/ $ grep -n -A50 -E 'context_switch(struct rq *rq, struct task_struct *prev, struct task_struct *next)' kernel/sched/core.c
2336:context_switch(struct rq *rq, struct task_struct *prev,
2337-                struct task_struct *next)
2338-{
2339-    struct mm_struct *mm, *oldmm;
2340-
2341-    prepare_task_switch(rq, prev, next);
2342-
2343-    mm = next->mm;
2344-    oldmm = prev->active_mm;
2345-    /*
2346-     * For paravirt, this is coupled with an exit in switch_to to
2347-     * combine the page table reload and the switch backend into
2348-     * one hypercall.
2349-     */
2350-    arch_start_context_switch(prev);
2351-
2352-    if (!mm) {
2353-        next->active_mm = oldmm;
2354-        atomic_inc(&oldmm->mm_count);
2355-        enter_lazy_tlb(oldmm, next);
2356-    } else
2357-        switch_mm(oldmm, mm, next);
2358-
2359-    if (!prev->mm) {
2360-        prev->active_mm = NULL;
2361-        rq->prev_mm = oldmm;
2362-    }
2363-    /*
2364-     * Since the runqueue lock will be released by the next
2365-     * task (which is an invalid locking op but in the case
2366-     * of the scheduler it's an obvious special-case), so we
2367-     * do an early lockdep release here:
2368-     */
2369-    spin_release(&rq->lock.dep_map, 1, _THIS_IP_);
2370-
2371-    context_tracking_task_switch(prev, next);
2372-    /* Here we just switch the register state and the stack. */
2373-    switch_to(prev, next, prev);
2374-
2375-    barrier();
2376-    /*
2377-     * this_rq must be evaluated again because prev may have moved
2378-     * CPUs since it called schedule(), thus the 'rq' on its stack
2379-     * frame will be invalid.
2380-     */

```

在 `context_switch()` 中，Linux 先准备任务切换状态，再根据 `mm` 关系执行 `switch_mm()` 或处理活动地址空间，之后调用架构相关的 `switch_to(prev, next, prev)`。切换完成后还会执行 `finish_task_switch()`。地址空间处理和寄存器/内核栈切换各有职责，不能把整个进程切换简化为一条汇编指令。

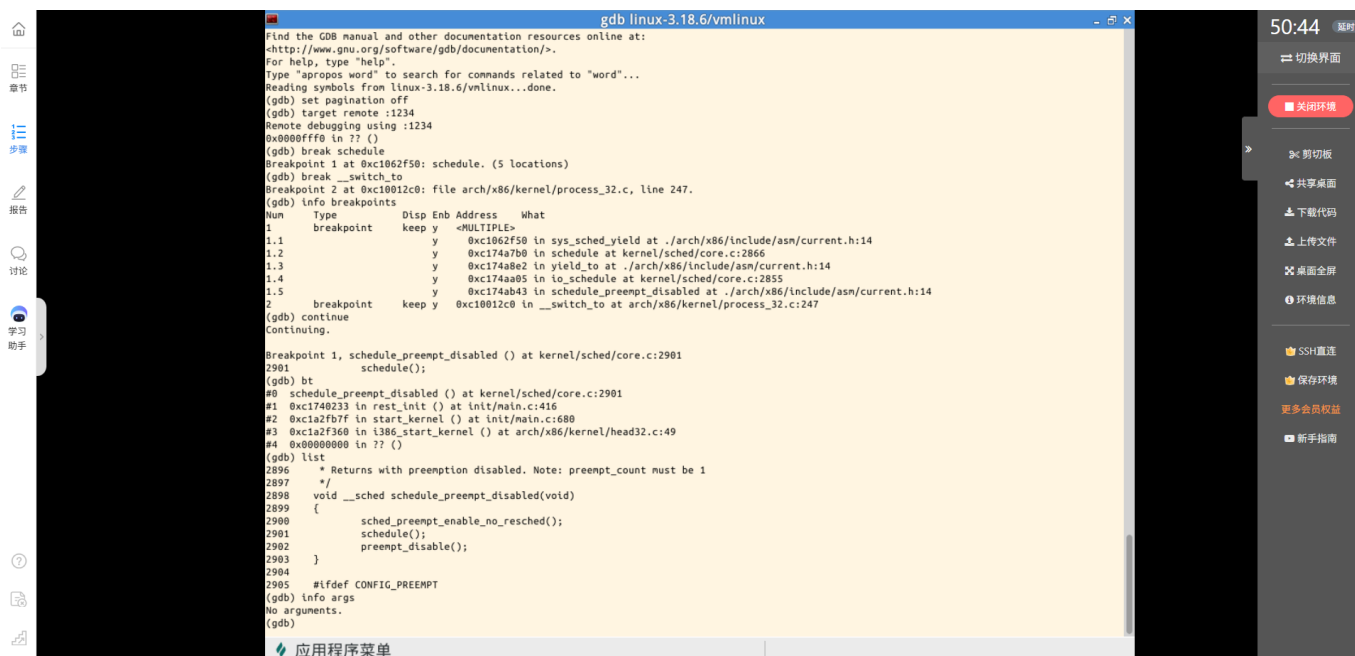
```
shiyanyou@6ab8fb486f8c024125cda4ea: ~/LinuxKernel/linux-3.18.6
2376- /*
2377-  * this_rq must be evaluated again because prev may have moved
2378-  * CPUs since it called schedule(), thus the 'rq' on its stack
2379-  * frame will be invalid.
2380-  */
2381- finish_task_switch(this_rq(), prev);
2382-}
2383-
2384-/*
2385- * nr_running and nr_context_switches:
2386- */
shiyanyou@linux-3.18.6/ $ grep -n -A48 -E '^#define switch_to(prev, next, last)' arch/x86/include/asm/switch_t
6.h
31:#define switch_to(prev, next, last)
32:do {
33- /*
34-  * Context-switching clobbers all registers, so we clobber
35-  * then explicitly, via unused output variables.
36-  * (EAX and EBP is not listed because EBP is saved/restored
37-  * explicitly for wchan access and EAX is the return value of
38-  * __switch_to())
39-  */
40- unsigned long ebx, ecx, edx, esi, edi;
41-
42- asm volatile("pushfl\n\t" /* save flags */
43-             "pushl %%ebp\n\t" /* save EBP */
44-             "movl %%esp,%%[prev_sp]\n\t" /* save ESP */
45-             "movl %%[next_sp],%%esp\n\t" /* restore ESP */
46-             "movl $if,%%[prev_ip]\n\t" /* save EIP */
47-             "pushl %%[next_ip]\n\t" /* restore EIP */
48-             __switch_canary
49-             "jmp __switch_to\n\t" /* regparm call */
50-             "i"
51-             "popl %%ebp\n\t" /* restore EBP */
52-             "popfl\n\t" /* restore flags */
53-
54-             /* output parameters */
55-             : [prev_sp] "m" (prev->thread.sp),
56-               [prev_ip] "m" (prev->thread.ip),
57-               "a" (last),
58-
59-             /* clobbered output registers: */
60-             "ab" (ebx), "ac" (ecx), "ad" (edx),
61-             "as" (esi), "ad" (edi)
62-
63-             : __switch_canary_oparam
64-
65-             :);
66-} while (0);
67-}
68-
69-/* regparm parameters for __switch_to(): */
70- [prev] "a" (prev),
71- [next] "d" (next)
72-
73- __switch_canary_iparam
74-
75- : /* reloaded segment registers */
76- "memory");
77-} while (0)
78-
79-#else /* CONFIG_X86_32 */
80-
81-
82-
83-
84-
85-
86-
87-
88-
89-
90-
91-
92-
93-
94-
95-
96-
97-
98-
99-
100-
101-
102-
103-
104-#define switch_to(prev, next, last) \
105-    "movq %%rsp,%%[threadrsp]([prev])\n\t" /* save RSP */ \
106-    "movq %%[threadrsp]([next]),%%rsp\n\t" /* restore RSP */ \
107-    "call __switch_to\n\t" \
108-    "movq __percpu_arg([current_task]),%%rsi\n\t" \
109-    __switch_canary \
110-    "movq %%[thread_info]([%rsi]),%%r8\n\t" \
111-    "movq %%rax,%%rdi\n\t" \
112-    "testl %%[tif_fork],%%[ti_flags]([%r8])\n\t" \
113-    "jnz ret_from_fork\n\t" \
114-    RESTORE_CONTEXT \
115-    : "a" (last) \
116-    : __switch_canary_oparam \
117-    : [next] "S" (next), [prev] "D" (prev), \
118-      [threadrsp] "l" (offsetof(struct task_struct, thread.sp)), \
119-      [ti_flags] "l" (offsetof(struct thread_info, flags)), \
120-      [tif_fork] "l" (_TIF_FORK), \
121-      [thread_info] "l" (offsetof(struct task_struct, stack)), \
122-      [current_task] "n" (current_task) \
123-    : __switch_canary_iparam \
124-    : "memory", "cc" __EXTRA_CLOBBER)
125-
126-
127-#endif /* CONFIG_X86_32 */
128-
129-#endif /* _ASM_X86_SWITCH_TO_H */
shiyanyou@linux-3.18.6/ $ grep -n -A32 -B1 'work_pending:' arch/x86/kernel/entry_32.S [19:24:36]
593:work_pending:
594- testb $TIF_NEED_RESCHED, %cl
595- jz work_notifysig
596:work_resched:
597- call schedule
598- LOCKDEP_SYS_EXIT
599- DISABLE_INTERRUPTS(CLBR_ANY) # make sure we don't miss an interrupt
600-                               # setting need_resched or sigpending
601-                               # between sampling and the iret
602- TRACE_IRQS_OFF
603- movl TI_FLAGS(%ebp), %ecx
604- andl $TIF_WORK_MASK, %ecx # is there any work to be done other
605-                               # than syscall tracing?
606- jz restore_all
607- testb $TIF_NEED_RESCHED, %cl
608- jnz work_resched
609-
610:work_notifysig:
611-                               # deal with pending signals and
612-                               # notify-resume requests
613-#ifdef CONFIG_VM86
614- testl $X86_EFLAGS_VM, PT_EFLAGS(%esp)
615- movl %esp, %eax
616- jne work_notifysig_vm86 # returning to kernel-space or
617-                               # vm86-space
618-1:
619- movl %esp, %eax
620-#endif
621- TRACE_IRQS_ON
622- ENABLE_INTERRUPTS(CLBR_NONE)
623- movb PT_CS(%esp), %bl
624- andb $SEGMENT_RPL_MASK, %bl
625- cmpl $USER_RPL, %bl
shiyanyou@linux-3.18.6/ $
```

```
shiyanyou@6ab8fb486f8c024125cda4ea: ~/LinuxKernel/linux-3.18.6
55- /* input parameters: */
56- : [next_sp] "n" (next->thread.sp),
57- : [next_ip] "n" (next->thread.ip),
58-
59- /* regparm parameters for __switch_to(): */
60- [prev] "a" (prev),
61- [next] "d" (next)
62-
63- __switch_canary_iparam
64-
65- : /* reloaded segment registers */
66- "memory");
67-} while (0)
68-
69-#else /* CONFIG_X86_32 */
70-
71-
72-
73-
74-
75-
76-
77-
78-
79-
80-
81-
82-
83-
84-
85-
86-
87-
88-
89-
90-
91-
92-
93-
94-
95-
96-
97-
98-
99-
100-
101-
102-
103-
104-#define switch_to(prev, next, last) \
105-    "movq %%rsp,%%[threadrsp]([prev])\n\t" /* save RSP */ \
106-    "movq %%[threadrsp]([next]),%%rsp\n\t" /* restore RSP */ \
107-    "call __switch_to\n\t" \
108-    "movq __percpu_arg([current_task]),%%rsi\n\t" \
109-    __switch_canary \
110-    "movq %%[thread_info]([%rsi]),%%r8\n\t" \
111-    "movq %%rax,%%rdi\n\t" \
112-    "testl %%[tif_fork],%%[ti_flags]([%r8])\n\t" \
113-    "jnz ret_from_fork\n\t" \
114-    RESTORE_CONTEXT \
115-    : "a" (last) \
116-    : __switch_canary_oparam \
117-    : [next] "S" (next), [prev] "D" (prev), \
118-      [threadrsp] "l" (offsetof(struct task_struct, thread.sp)), \
119-      [ti_flags] "l" (offsetof(struct thread_info, flags)), \
120-      [tif_fork] "l" (_TIF_FORK), \
121-      [thread_info] "l" (offsetof(struct task_struct, stack)), \
122-      [current_task] "n" (current_task) \
123-    : __switch_canary_iparam \
124-    : "memory", "cc" __EXTRA_CLOBBER)
125-
126-
127-#endif /* CONFIG_X86_32 */
128-
129-#endif /* _ASM_X86_SWITCH_TO_H */
shiyanyou@linux-3.18.6/ $ grep -n -A32 -B1 'work_pending:' arch/x86/kernel/entry_32.S [19:24:36]
593:work_pending:
594- testb $TIF_NEED_RESCHED, %cl
595- jz work_notifysig
596:work_resched:
597- call schedule
598- LOCKDEP_SYS_EXIT
599- DISABLE_INTERRUPTS(CLBR_ANY) # make sure we don't miss an interrupt
600-                               # setting need_resched or sigpending
601-                               # between sampling and the iret
602- TRACE_IRQS_OFF
603- movl TI_FLAGS(%ebp), %ecx
604- andl $TIF_WORK_MASK, %ecx # is there any work to be done other
605-                               # than syscall tracing?
606- jz restore_all
607- testb $TIF_NEED_RESCHED, %cl
608- jnz work_resched
609-
610:work_notifysig:
611-                               # deal with pending signals and
612-                               # notify-resume requests
613-#ifdef CONFIG_VM86
614- testl $X86_EFLAGS_VM, PT_EFLAGS(%esp)
615- movl %esp, %eax
616- jne work_notifysig_vm86 # returning to kernel-space or
617-                               # vm86-space
618-1:
619- movl %esp, %eax
620-#endif
621- TRACE_IRQS_ON
622- ENABLE_INTERRUPTS(CLBR_NONE)
623- movb PT_CS(%esp), %bl
624- andb $SEGMENT_RPL_MASK, %bl
625- cmpl $USER_RPL, %bl
shiyanyou@linux-3.18.6/ $
```

```
shiyanyou@6ab8fb486f8c024125cda4ea: ~/LinuxKernel/linux-3.18.6
123- [current_task] "n" (current_task)
124- __switch_canary_iparam
125- : "memory", "cc" __EXTRA_CLOBBER)
126-
127-#endif /* CONFIG_X86_32 */
128-
129-#endif /* _ASM_X86_SWITCH_TO_H */
shiyanyou@linux-3.18.6/ $ grep -n -A32 -B1 'work_pending:' arch/x86/kernel/entry_32.S [19:24:36]
593:work_pending:
594- testb $TIF_NEED_RESCHED, %cl
595- jz work_notifysig
596:work_resched:
597- call schedule
598- LOCKDEP_SYS_EXIT
599- DISABLE_INTERRUPTS(CLBR_ANY) # make sure we don't miss an interrupt
600-                               # setting need_resched or sigpending
601-                               # between sampling and the iret
602- TRACE_IRQS_OFF
603- movl TI_FLAGS(%ebp), %ecx
604- andl $TIF_WORK_MASK, %ecx # is there any work to be done other
605-                               # than syscall tracing?
606- jz restore_all
607- testb $TIF_NEED_RESCHED, %cl
608- jnz work_resched
609-
610:work_notifysig:
611-                               # deal with pending signals and
612-                               # notify-resume requests
613-#ifdef CONFIG_VM86
614- testl $X86_EFLAGS_VM, PT_EFLAGS(%esp)
615- movl %esp, %eax
616- jne work_notifysig_vm86 # returning to kernel-space or
617-                               # vm86-space
618-1:
619- movl %esp, %eax
620-#endif
621- TRACE_IRQS_ON
622- ENABLE_INTERRUPTS(CLBR_NONE)
623- movb PT_CS(%esp), %bl
624- andb $SEGMENT_RPL_MASK, %bl
625- cmpl $USER_RPL, %bl
shiyanyou@linux-3.18.6/ $ [19:24:41]
```

三、GDB：从启动期调度入口到真实任务切换

GDB 加载与 QEMU 启动镜像配套的 `vmlinux`，连接 QEMU 的 GDB stub，并设置 `schedule` 与 `__switch_to` 断点。GDB 显示 `break schedule` 解析为多个位置；继续运行后，第一次停在 `schedule_preempt_disabled()` 中调用 `schedule()` 的位置 (`kernel/sched/core.c:2901`)。调用栈依次回到 `rest_init()`、`start_kernel()` 和 `i386_start_kernel()`。因此这次命中来自 Linux 启动期路径，不是 MenuOS 用户命令，也不是一次已观察到的系统调用退出。



```

gdb linux-3.18.6/vmlinux
Find the GDB manual and other documentation resources online at:
~http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from linux-3.18.6/vmlinux...done.
(gdb) set pagination off
(gdb) target remote :1234
Remote debugging using :1234
0x00000000 in ?? ()
(gdb) break schedule
Breakpoint 1 at 0xc1062f50: schedule. (5 locations)
(gdb) break __switch_to
Breakpoint 2 at 0xc10012c0: file arch/x86/kernel/process_32.c, line 247.
(gdb) info breakpoints
Num Type Disp Enb Address What
1 breakpoint keep y <MULTIPLE>
1.1 breakpoint y 0xc1062f50 in sys_sched_yield at ./arch/x86/include/asm/current.h:14
1.2 breakpoint y 0xc174a7b0 in schedule at kernel/sched/core.c:2866
1.3 breakpoint y 0xc174a8e2 in yield_to at ./arch/x86/include/asm/current.h:14
1.4 breakpoint y 0xc174a905 in to_schedule at kernel/sched/core.c:2855
1.5 breakpoint y 0xc174ab43 in schedule_preempt_disabled at ./arch/x86/include/asm/current.h:14
2 breakpoint keep y 0xc10012c0 in __switch_to at arch/x86/kernel/process_32.c:247
(gdb) continue
Continuing.

Breakpoint 1, schedule_preempt_disabled () at kernel/sched/core.c:2901
2901 schedule();
(gdb) bt
#0 schedule_preempt_disabled () at kernel/sched/core.c:2901
#1 0xc1740233 in rest_init () at init/main.c:416
#2 0xc1a2fb7f in start_kernel () at init/main.c:680
#3 0xc1a2f360 in i386_start_kernel () at arch/x86/kernel/head32.c:49
#4 0x00000000 in ?? ()
(gdb) list
2896 * Returns with preemption disabled. Note: preempt_count must be 1
2897 */
2898 void __sched schedule_preempt_disabled(void)
2899 {
2900     sched_preempt_enable_no_resched();
2901     schedule();
2902     preempt_disable();
2903 }
2904
2905 #ifdef CONFIG_PREEMPT
(gdb) info args
No arguments.
(gdb)
应用程序菜单

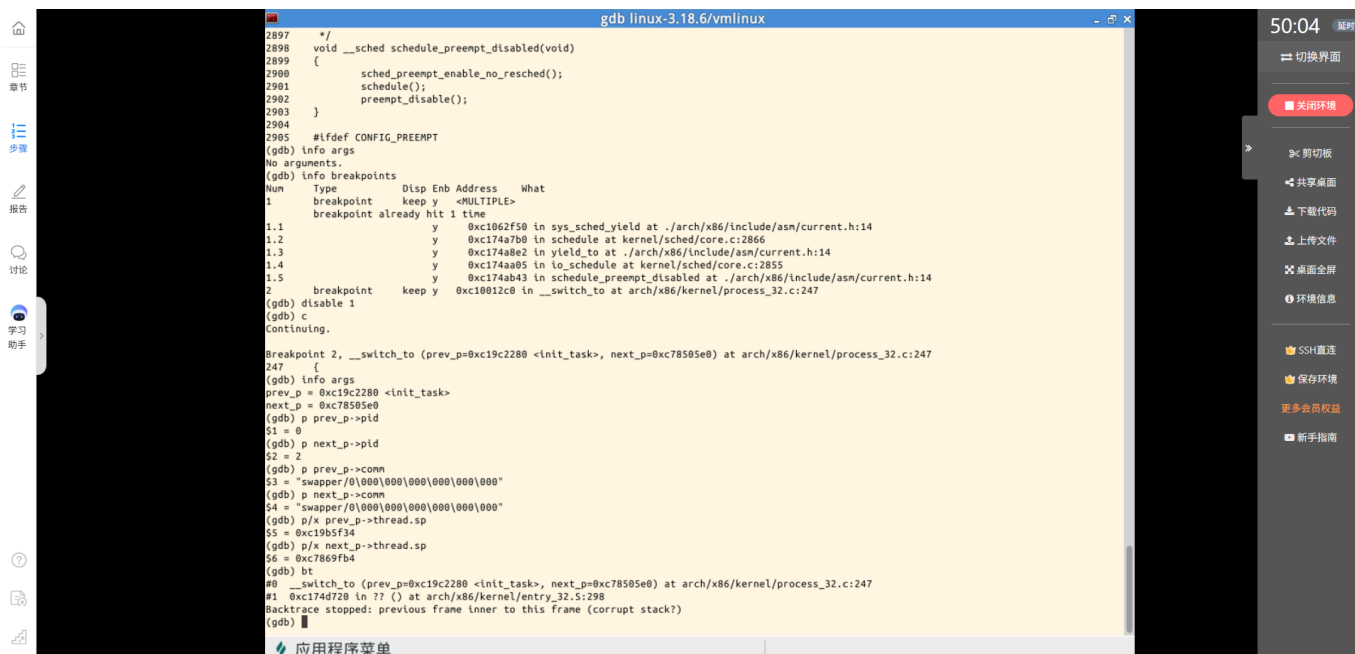
```

我禁用 `schedule` 断点后继续运行，GDB 在 `__switch_to()` 处停下。参数和结构体字段显示：

- `prev_p` 指向 `init_task`，PID 为 0；
- `next_p` 的 PID 为 2；
- `prev_p->thread.sp` 为 `0xc19b5f34`；
- `next_p->thread.sp` 为 `0xc7869fb4`。

PID 0 到 PID 2 的不同，连同 `__switch_to(prev_p, next_p)` 的断点，证明启动过程中确实从一个任务切换到另一个任务。它不是“只命中 `schedule()`”，也不是 MenuOS 用户进程之间的切换。GDB 对 `comm` 字段的显示不可靠，因此这里依据可读的 PID、指针和栈字段陈述结果，不用任务名补足结论。

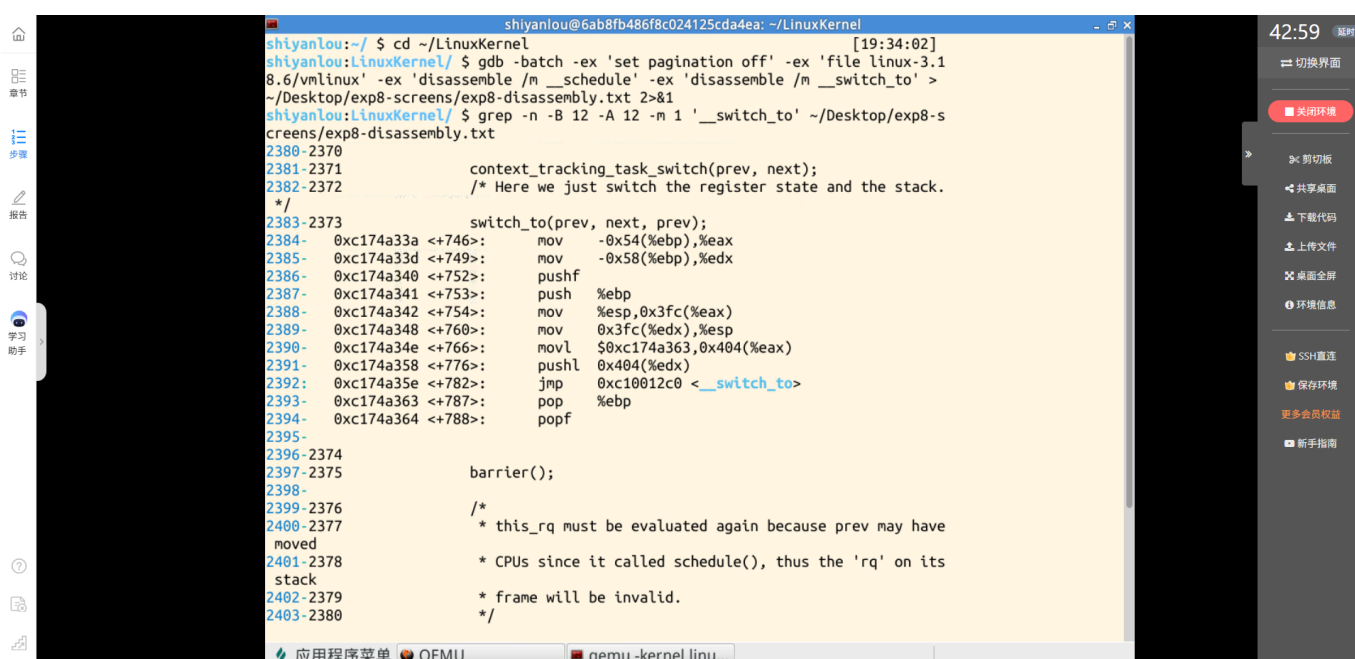
GDB 在该断点处的 `bt` 提示回溯在此停止，并出现栈完整性警告。任务切换正在改变当前内核栈和执行位置；不能把跨过 `switch_to` 前后的栈帧当作一条普通、连续的 C 调用链。这个警告说明调试器无法按普通栈帧规则继续展开，不会推翻参数中观察到的两个任务 PID。



四、switch_to：怎样保存旧栈并接上新栈

Linux 3.18.6 x86 32 位的 `switch_to` 宏把旧任务的内核执行现场留在它自己的栈和 `thread` 字段里，再从新任务的栈与继续位置恢复执行。源码和本次反汇编共同显示了这个关键次序：

1. `pushfl` 和 `pushl %ebp`：把标志寄存器和帧指针保存在当前任务的内核栈上。
2. `movl %esp, prev->thread.sp`：记录旧任务当前的栈指针。
3. `movl next->thread.sp, %esp`：把 CPU 的栈指针换到新任务的内核栈。
4. `movl $1f, prev->thread.ip`：把旧任务恢复后要继续执行的本地标签地址保存到 `prev->thread.ip`。
5. `pushl next->thread.ip`：把新任务的继续地址压入新栈。
6. `jmp __switch_to`：转入架构相关的切换代码；返回后由标签 1 处的 `popl %ebp`、`popfl` 恢复新任务自己的帧指针和标志寄存器。



反汇编中，这些指令出现在 `context_switch()` 内联展开的位置：先将参数指针装入寄存器，再保存和装载 ESP，保存旧的继续地址、压入新的继续地址，最后跳转到 `__switch_to`。截图里的结构偏移和指令地址属于

这次 Linux 3.18.6 构建，不应当当作所有内核版本或架构的固定数值。

任务切换保存的是内核执行上下文。一个已经运行过的任务稍后再次被选中时，会从自己先前保存的栈和继续位置接着执行；它并不会重新从内核入口开始。新创建任务的首次运行入口则需要创建时专门准备，例如实验六分析过的 `ret_from_fork`。本次 PID 0→2 的跟踪没有读取 `next` 的 `thread.ip`，所以该字段的实际值不写成本次动态观察结果。

五、switch_to 与中断现场不是同一件事

系统调用或中断入口会建立返回所需的寄存器现场和内核栈帧；在 x86 32 位路径中，相关现场通过入口汇编 `pt_regs` 等机制保存和恢复。用户任务进入内核处理系统调用时，内核仍在该任务的执行上下文中运行。

若内核判断需要重新调度，可能在系统调用/异常返回的安全位置调用调度器。上图的 `work_pending` 展示了一个条件检查：只有 `TIF_NEED_RESCHED` 置位时才进入 `work_resched` 并调用 `schedule`。这不是“每次系统调用都会切换”，也不是“中断一发生就换任务”。

若最终切到另一个任务，`switch_to` 保存当前任务的内核栈和继续位置，让 CPU 转到另一个任务的内核栈。原任务此前由系统调用/中断入口建立的返回路径仍留在它自己的栈上；当它以后再次获得 CPU，内核会从该任务自己的继续位置恢复，之后再完成返回工作。入口现场保存/恢复与任务栈切换相互配合，但用途不同。

六、我对 Linux 系统一般执行过程的理解

Linux 系统一般执行过程可以这样理解：

1. 用户进程在用户态执行指令。
2. 系统调用、异常或中断发生后，CPU 和内核入口代码保存必要的返回现场，转入内核态处理。
3. 内核完成请求；若任务需要等待，它可以主动进入调度器；若有更合适任务，内核也可能设置重调度标志，等待安全点处理。
4. 调度器选择下一个可运行任务。若仍是当前任务，就继续原执行现场；若选择了另一任务，内核按架构规则切换地址空间（需要时）、内核栈和寄存器/继续位置。
5. 新任务在自己的内核现场继续运行。它完成内核工作后，可能返回用户态，也可能继续执行内核线程工作；以后被切出的任务可以从自己的保存点恢复。

因此“进入内核”“调用了调度器”“真正切换了任务”和“返回用户态”是相互关联但不同的阶段。中断和系统调用是进入内核的常见入口，调度是内核根据状态和重调度条件作出的选择，`switch_to` 则负责在两个任务的内核执行现场之间衔接。新进程的 `ret_from_fork` 首次运行以及 `execve` 重建用户态入口，是一般返回路径中的特殊情况。

七、结论与证据边界

本次源码检索产生了大量候选行；它提示我必须继续按调用上下文分类，而不能把 `grep` 行数写成实际调度次数。源码显示了 `work_pending` 的条件调度路径和 `context_switch()` 调用 `switch_to` 的位置；GDB 实际观察到启动期从 PID 0 到 PID 2 的任务切换；反汇编片段显示本次构建中 ESP 和继续位置的切换指令。三类材料分别回答潜在调用点、一次真实运行路径和架构现场切换方式，不能互相替代。

本次未观察 MenuOS 用户进程之间的切换，未逐行人工核完 `grep` 的全部候选调用点，也没有动态读取 `next` 的 `thread.ip`。因此博客结论限定在已保存的源码和 GDB 证据范围内；这些项目不作为已完成的动态观察来描述。

参考资料

1. 《庖丁解牛 Linux 操作系统分析》，第八章 8.2、8.3.1、8.3.2；第九章 9.1.1、9.1.4、9.2.3。
 2. Linux 3.18.6 `kernel/sched/core.c`：调度器与 `context_switch`
 3. Linux 3.18.6 `arch/x86/include/asm/switch_to.h`：x86 `switch_to` 宏
 4. Linux 3.18.6 `arch/x86/kernel/process_32.c`：x86-32 `__switch_to`
 5. Linux 3.18.6 `arch/x86/kernel/entry_32.S`：系统调用与 `work_pending` 返回路径
-

发布到 CSDN 后，在此处补充公开博客 URL；当前 Markdown 是草稿，未发布。